

AETHER

Technical Architecture & AI Pipeline

How the AI Tutor That Never Forgets actually works

Made by **Gurnoor Tamber**

Document Purpose: This document explains the internal architecture of Aether — the AI-powered personalized tutoring platform. It covers the system design, AI pipeline (RAG), data flow, and key technical decisions for developers and technical stakeholders.

Stack Overview: Next.js 16 · React 19 · TypeScript · Tailwind CSS · Supabase (PostgreSQL + Auth + Storage) · Google Gemini API · Deepgram · pgvector · Pyodide · KaTeX · Mermaid

Repository: Private · Monorepo · Turbopack · ESLint 9

Version 1.0 — July 2026

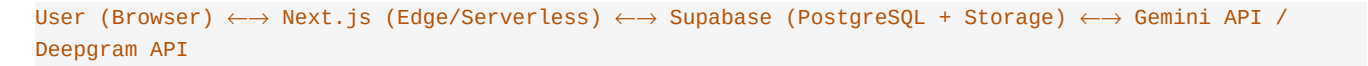
Contents

1. System Architecture Overview
2. Frontend Architecture
3. Backend & Database Schema
4. The RAG Pipeline (Core AI Flow)
 - 4.1 Document Upload & Storage
 - 4.2 Text Extraction
 - 4.3 Chunking Strategy
 - 4.4 Embedding Generation
 - 4.5 Vector Search (pgvector)
 - 4.6 Context Assembly & Prompt Construction
 - 4.7 Streaming Response Generation
 - 4.8 Client-Side Rendering
5. Voice Tutor Architecture (Deepgram)
6. Quiz & Challenge Engines
7. Music Generation Pipeline
8. Beta Gating & Usage Limits
9. Key Technical Decisions

1. System Architecture Overview

Aether follows a modern JAMstack architecture with server-rendered React (Next.js App Router) backed by Supabase for database, authentication, and file storage. The AI layer is powered entirely by Google Gemini models, with Deepgram handling voice processing.

High-Level Data Flow



Key architectural properties:

- Server-rendered React via Next.js App Router with streaming SSR
- Serverless API routes for Chat, TTS, Knowledge ingestion, Quiz generation, etc.
- Row-Level Security (RLS) on every database table — users see only their own data
- Service role for privileged operations (usage increments, beta approval)
- All AI calls are server-side — API keys never reach the client
- Pyodide runs Python 100% in-browser for code challenges — zero server cost

Tech Stack Summary

Layer	Technology
Framework	Next.js 16.2.10 (App Router)
Language	TypeScript 5
UI	React 19.2.4, Tailwind CSS v4
Fonts	Inter, JetBrains Mono, Syne
Icons	lucide-react
Database	Supabase PostgreSQL + pgvector
Auth	Supabase Auth (email, magic link, OAuth)
Storage	Supabase Storage
AI Chat	Google Gemini Flash 2.0
Embeddings	Google Gemini Embedding
Voice STT	Deepgram (real-time)
Voice TTS	Web Speech API
Math	KaTeX
Diagrams	Mermaid.js
Graphing	Desmos API v1.9
Code Exec	Pyodide (WebAssembly)

Music	HeartMuLa Studio + MusicGen
PDF Parsing	pdf-parse
PPTX Parsing	JSZip + XML
Deployment	Vercel

2. Frontend Architecture

The frontend is a Next.js 16 App Router application with React 19 and TypeScript. The design system is dark-themed glassmorphism with cyber-yellow (#FDE047) accents.

Route Structure

Route	Purpose
/	Landing page — hero, features, pricing, beta signup
/dashboard	Main dashboard — stats, recent docs, quick chat
/hub	Session hub — create/search/delete learning sessions
/onboarding	Conversational 7-step onboarding flow
/[session]/dashboard	Per-session dashboard with modules overview
/[session]/chat	AI chat with RAG, streaming, markdown, KaTeX, Mermaid
/[session]/voice-tutor	Real-time voice conversation (Deepgram)
/[session]/roadmap	Visual module sequence with progress
/[session]/quizzes	Generate and take AI quizzes
/[session]/knowledge	Document upload and management
/[session]/progress	Analytics — mastery, XP, strengths, milestones
/[session]/music	AI focus music generation and player
/[session]/challenges	Coding and math challenge hub
/[session]/challenge-code/[id]	In-browser Python code challenge
/[session]/challenge-math/[id]	Math challenge with Desmos graphing

Key UI Architecture Decisions

- Three-panel layout within sessions: SidebarLeft | Main Content | SidebarRight
- Auto-collapsible hero headers on roadmap/quizzes/progress pages — save screen space, expand on hover
- Streaming responses via ReadableStream with custom SSE-like parser — shows thinking status for every stage (searching, analyzing, generating)
- Structured block parsing in chat — AI can emit visual explanations, step-by-step walkthroughs, interactive demos, code blocks, and mermaid diagrams as distinct rendered sections
- Module context mode — chat can be scoped to a specific roadmap module, showing module info and lesson list

3. Backend & Database Schema

The backend uses Supabase PostgreSQL with Row-Level Security. Key tables:

Core Tables

Table	Purpose
user_profiles	User name, onboarding status, preferences (JSON)
beta_requests	Beta signup queue — email + approved boolean
user_usage	Usage counters per user (chat, quiz, voice, challenge)
sessions	Learning sessions — title, subject, slug, user_id
session_roadmap_modules	Roadmap modules — title, status, lessons (JSON), learning objectives
conversations	Chat conversations — title, session_id, user_id
chat_messages	Individual messages — role, content, conversation_id
session_quizzes	Quiz records — questions (JSON), score, module_id
documents	Uploaded docs — filename, type, status, storage path
document_chunks	Text chunks with pgvector embeddings
ai_memories	Persisted AI outputs (challenges, etc.)
progress_tracking	Mastery levels per subject per user

Key Database Features

- pgvector extension for cosine similarity search via match_document_chunks RPC
- Row-Level Security (RLS) on every user-facing table — user_id must match auth.uid()
- increment_usage() SECURITY DEFINER function — service role bypasses RLS for counter updates
- delete_user_sessions() SECURITY DEFINER function — cleanup on account deletion

4. The RAG Pipeline (Core AI Flow)

Retrieval-Augmented Generation (RAG) is the heart of Aether. It enables the AI to answer questions based on the user's own uploaded documents, rather than general knowledge. Below is the complete pipeline, stage by stage.

4.1 Document Upload & Storage

When a user uploads a file through the Knowledge page, the following sequence occurs:

- File is sent via FormData POST to /api/knowledge/image (for images) or the ingest pipeline
- File is uploaded to Supabase Storage under the user's session folder
- A record is inserted into the documents table with metadata (filename, size, type, status='pending')
- Supported formats: PDF (.pdf), PowerPoint (.pptx), Images (.png, .jpg, .jpeg, .webp), YouTube links, Google Drive links
- YouTube: /api/knowledge/youtube extracts transcript via youtube-transcript API
- Google Drive: /api/knowledge/gdrive downloads the file via Drive link, then processes as normal

The upload UI shows real-time progress per file across 5 stages:

pending → uploading → extracting → chunking → embedding → done

4.2 Text Extraction

Each file type uses a different extraction strategy:

- **PDF:** pdf-parse library extracts text page by page. Preserves paragraph structure, discards layout formatting.
- **PPTX:** File is unzipped via JSZip. Each slide XML is parsed for text content in text runs. Combines all slide text sequentially.
- **Images:** Sent to Gemini Vision API for OCR — the model transcribes visible text from the image.
- **YouTube:** youtube-transcript API fetches the video's closed caption / transcript as timed text.
- **Google Drive:** File is downloaded, then routed to the appropriate extractor based on extension.

Extracted raw text is stored temporarily for the next stage. The document status updates to 'extracting' during this phase.

4.3 Chunking Strategy

Raw document text is too long for a single embedding (and would lose granularity). We split it into overlapping chunks:

- Chunk size: 500-1000 characters per chunk
- Overlap: 100 characters between consecutive chunks
- Overlap ensures concepts that span chunk boundaries aren't lost
- Chunks are created by splitting on paragraph breaks first, then splitting oversized paragraphs on sentence boundaries
- Each chunk is stored as a row in document_chunks with: document_id, chunk_index, content, and a NULL embedding column (filled next step)
- Document status updates to 'chunking'

4.4 Embedding Generation

Every chunk is converted into a vector embedding — a numerical representation of its semantic meaning:

- Each chunk is sent to the **Gemini Embedding model** via the Google AI SDK
- The model returns a 768-dimensional vector (array of floats)
- Output dimensionality is model-dependent — Gemini uses 768 dimensions
- The vector is stored in the embedding column of document_chunks (pgvector data type)

- An IVFFlat index on the embedding column accelerates nearest-neighbor search
- Document status updates to 'done' when all chunks are embedded

The entire pipeline (upload → extract → chunk → embed) runs synchronously in the API route. For large documents, this can take 10-60 seconds. The UI shows real-time progress so the user knows the system is working.

4.5 Vector Search (pgvector)

At query time, when a user asks a question in chat, the following search process runs:

- The user's question text is sent to the same Gemini Embedding model → query vector
- A PostgreSQL function `match_document_chunks` is called:

```
function match_document_chunks(  
  query_embedding vector(768),  
  match_threshold float,  
  match_count int,  
  session_id uuid  
) returns table(id bigint, content text, similarity float)  
language sql stable  
as $$  
  select dc.id, dc.content,  
    1 - (dc.embedding <=> query_embedding) as similarity  
  from document_chunks dc  
  join documents d on d.id = dc.document_id  
  where d.session_id = session_id  
    and dc.embedding is not null  
    and 1 - (dc.embedding <=> query_embedding) > match_threshold  
  order by dc.embedding <=> query_embedding  
  limit match_count;  
$$;
```

- Uses `<=>` operator (cosine distance) — converts to similarity via `1 - distance`
- Filters by `session_id` — only searches documents belonging to this session
- Threshold: typically 0.7 — chunks below this similarity are discarded
- Top 5-10 chunks are returned, ordered by similarity descending
- Search is scoped to the current session — documents from other sessions are invisible

4.6 Context Assembly & Prompt Construction

Retrieved chunks are assembled into a structured prompt:

- The system prompt defines Aether's behavior: friendly tutor, uses analogies, references documents, can use LaTeX and Mermaid
- Retrieved chunks are formatted as a 'Retrieved Context' section with source document references
- Conversation history (last N messages) is appended for multi-turn continuity
- If module context is active (chatting from a roadmap module), the module's title, description, learning objectives, and lesson list are injected
- The user's current question is appended at the end

The full prompt typically consumes 4,000-12,000 tokens depending on context length.

4.7 Streaming Response Generation

The assembled prompt is sent to Gemini Flash with streaming enabled:

- POST to Gemini API with stream: true
- Response is a ReadableStream of chunks
- Each chunk from Gemini is forwarded to the client as an SSE event
- Three event types: status (thinking indicator), chunk (text token), done (final message ID)
- Status events are emitted before the first token — 'Analyzing documents...', 'Searching knowledge base...', 'Formulating response...', etc.
- Chunk events accumulate into a full response text on the client
- On 'done', the accumulated text is saved to chat_messages with the final message ID

The response is streamed through Vercel Edge functions for minimal latency. Connection stays open until the full response is delivered.

4.8 Client-Side Rendering Pipeline

As the stream arrives, the client parses and renders in real time:

- Raw SSE data lines are parsed — type field determines handler
- Accumulated text is split into structured blocks by the parseStructuredBlocks() function
- Block detection logic:

```
// Detected by scanning for keywords in the text:
***Visual Explanation*** or ***Visualizing*** → type='visual'
***Interactive Demo*** or ***Try It Yourself*** → type='interactive'
***Step-by-Step*** or ***Walkthrough*** → type='stepbystep'
``` fences with 'mermaid' → type='mermaid'
``` fences with any other lang → type='code'
Everything else → type='text'
```

- Each block type renders differently:
- **text**: renderMarkdown() converts markdown → HTML. Sub-steps: escape HTML chars → split by paragraphs → detect headings, blockquotes, lists, HRs → apply inlineMarkdown() for bold, italic, code, links, LaTeX → dangerouslySetInnerHTML
- **LaTeX**: inlineMarkdown() applies two regex passes: $\\$\\$\\$\\$$ → display math (KaTeX renderToString with displayMode:true), then $\\$\\$\\$$ → inline math. Errors fall back to raw LaTeX text.
- **mermaid**: MermaidBlock component initializes mermaid module once (mermaid.initialize()), calls mermaid.render(id, chartText) to get SVG string, then sets innerHTML on a div. Hidden until render completes.
- **code**: Pre block with monospace font, yellow text, dark background, overflow-x-auto scroll
- **visual**: Glass card with cyan header icon + 'Visual Explanation' label
- **interactive**: Glass card with yellow header icon + 'Interactive Demo' label
- **stepbystep**: Glass card with purple header icon + 'Step-by-Step' label
- Each message bubble shows: role (user/assistant), timestamp, structured content
- User messages: right-aligned, subtle glass card, no special rendering
- Assistant messages: left-aligned with Aether avatar (yellow sparkle icon), full structured rendering

5. Voice Tutor Architecture (Deepgram)

The Voice Tutor provides real-time conversational tutoring using Deepgram for speech-to-text and the browser's Web Speech API for text-to-speech.

Data Flow

- User speaks into microphone → browser MediaStream → Deepgram real-time STT API
- Deepgram returns transcribed text → sent to /api/chat (same endpoint as text chat) → receives streaming response
- Response text → Web Speech API (SpeechSynthesisUtterance) → spoken back to user
- Full conversation history is saved to chat_messages in Supabase
- When user returns to text chat, all voice conversation history is visible

UI Components

- Center: pulsing yellow circle with glow effect — pulses faster when Aether is speaking
- 13 animated waveform bars that dance when connected
- Call timer (MM:SS) in header
- User transcription panel (left) — shows transcribed text with character avatar
- AI response panel (right) — shows Aether's spoken response with typing cursor
- Footer controls: mic toggle (yellow=on, red=muted), volume slider, speaker grid toggle, output mute, red 'End Session' button

6. Quiz & Challenge Engines

Quiz Engine

- Generation: POST /api/quizzes/generate with session_id + module_id → Gemini generates 5 MCQ questions with 4 options each, correct answer index, and explanation
- Storage: session_quizzes table — questions stored as JSON array, linked to module via module_id
- Taking: client-side state machine (list → taking → results). User selects answer per question, can skip, reviews all at end
- Scoring: percentage + XP calculation. Per-question review shows user's answer vs correct answer with AI explanation
- Results saved to progress_tracking for mastery calculation

Code Challenge Engine

- Generation: POST /api/challenges/generate with subject + topic + type:'code' → Gemini generates a coding problem with starter code, test cases, and solution
- Storage: sessionStorage + ai_memories table (JSON serialized)
- Execution: Pyodide (Python compiled to WebAssembly) runs in the browser. No server-side execution. Code is evaluated in a sandboxed Python interpreter.
- UI: multiple code cells, each with run button. Output displayed per cell. Test cases shown at bottom.

Math Challenge Engine

- Generation: similar to code challenges but type:'math' → Gemini generates problem with LaTeX expression and answer

- Desmos Integration: DesmosGraph component loads Desmos API v1.9 from CDN, creates a Calculator instance in a div. Challenge's LaTeX expression is plotted. User can interact with the graph (zoom, pan, click).
- Answer checking: case-insensitive, whitespace-normalized string comparison. Hints shown on wrong answer.

7. Music Generation Pipeline

- Two provider options: HeartMuLa Studio (primary) and MusicGen (via Colab API ngrok tunnel)
- User selects mood (Focused, Calm, Energetic, Dreamy, Dark) and instrument (Ambient Synth, Piano, Guitar, Drums, Strings, Bass, Flute, Saxophone)
- Optional: 'Enhance Prompt' button sends mood + instrument to Gemini → generates rich music prompt text
- Prompt is sent to the provider API → returns audio URL
- Track is saved to generated_tracks table
- Full music player via PlayerProvider React context: play/pause, seek, next/prev, queue management, volume
- Playlists: create, name, add/remove tracks

8. Beta Gating & Usage Limits

- Users sign up via landing page beta form → email inserted into beta_requests with approved=false
- Admin dashboard (local-admin/, not deployed) lets admins approve/reject/delete requests
- On sign in, server checks if user's email has approved=true in beta_requests
- Usage limits: enforced in lib/usage.ts — LIMITS = { chat: 10, quiz: 10, voice: 10, challenge: 10 }
- checkUsage() reads from user_usage table, incrementUsage() calls SECURITY DEFINER RPC to bypass RLS
- UsageIndicator component shows remaining count per type in the UI

9. Key Technical Decisions

Why pgvector over a dedicated vector DB?

Keeps infrastructure simple — no external Pinecone/Weaviate. Everything lives in Postgres with the rest of the data. One less network hop, one less API key. IVFFlat indexes are fast enough for <100K chunks.

Why Gemini over OpenAI?

Gemini Flash offers the best latency/cost tradeoff for streaming chat. Gemini Embedding is tightly integrated. The 1M-token context window (on Pro) allows massive context assembly without chunking.

Why Pyodide for code execution?

Zero server cost. Code runs entirely in the user's browser via WebAssembly. No sandboxing concerns, no cold starts, no timeouts. Limited to Python but sufficient for educational challenges.

Why Deepgram over alternatives?

Real-time streaming STT with low latency. The @deepgram/agents library provides turn-based conversation management out of the box. Competitive pricing for educational use.

Why local admin dashboard?

Security — the admin panel is not deployed or accessible via the public URL. It's a standalone Node server that connects to Supabase via `service_role` key. Starts with `start.bat`, only accessible on localhost.

Why KaTeX over MathJax?

KaTeX is ~10x faster. MathJax is more feature-complete but for inline math in chat messages, KaTeX's speed is critical. The synchronous `renderToString` API integrates cleanly with markdown parsing.

Chunk size of 500-1000 chars?

Large enough to contain a coherent concept, small enough that cosine similarity is meaningful. 100-char overlap prevents boundary issues. This is a standard range in production RAG systems.

SSE streaming vs WebSocket?

SSE is simpler — unidirectional `server→client` is all we need. No reconnection logic, no state synchronization. The client opens a `fetch` with `ReadableStream`, parses newline-delimited JSON events.

End of Document